

**Advanced Computer Systems Project 3 Report**

Ivan Cheng (ECSE)

Rensselaer Polytechnic Institute

ECSE 4320 Advanced Computer Systems

Dr. Tong. Zhang

December 15th, 2025

# Overview

This project evaluates the scalability of concurrent hash table designs under multi-threaded workloads. Two synchronization strategies are implemented being a coarse-grained global mutex and a striped locking approach. The goal is to quantify how synchronization granularity impacts throughput and scalability under lookup, insert, and mixed workloads, and to relate observed performance trends to contention, cache behavior, and Amdahl's Law.

## Experimental Methodology

### Hash Table Structure

The hash table is implemented using separate chaining where each bucket contains a dynamic list of key value pairs.

### Synchronization Strategies

#### Coarse-Grained Locking

The coarse grained implementation has all operations protected by a single global mutex. This design forces all threads to serialize when accessing the table.

#### Striped Locking

In the striped locking implementation the hash table buckets are partitioned across multiple locks. Each bucket maps to a single lock allowing concurrent access to different buckets. This reduces contention by limiting synchronization to only the subset of buckets being accessed which enables greater parallelism under multi-threaded workloads.

### Workload Generation

#### Key Generation

Keys were generated using a fixed seed pseudo random number generator to ensure reproducibility across runs.

#### Lookup-Only Workload

All operations are lookups on pre inserted keys.

#### Insert-Only Workload

All operations insert new key value pairs into the table.

#### Mixed Workload (70/30)

Each operation is selected probabilistically: 70% lookup, 15% insert, and 15% erase.

## Benchmark Configuration

### Thread Counts

Experiments were conducted using 1, 2, 4, 8, and 16 threads.

### Table Sizes

Three table sizes were evaluated:

- 10,000 keys
- 100,000 keys
- 1,000,000 keys

### Lock Count (Striped)

The striped locking implementation used 1,024 locks.

## Benchmarking Method

### Prefill Phase

For lookup and mixed workloads, the hash table was prefilled with the target number of keys before timing began.

### Timing

Each benchmark run executed for a fixed duration of 2 seconds.

### Metric Collection

Performance was measured as total operations completed per second (ops/s). Speedup was computed relative to the single thread baseline.

### Repetitions

Each configuration was executed three times with identical parameters, and results were averaged to reduce measurement noise.

## Machine Model and Environment

- Intel i7-11850H
- Base clock: 2.5 GHz
- AVX-512 available
- Theoretical FP32 peak: 640 GFLOP/S
- DDR4-3200 peak bandwidth  $\approx$  51.2 GB/s
- Environment: WSL2 Linux subsystem

## Commands used in experiment

- make clean
- make -j
- bash scripts/run\_all.sh

## Results

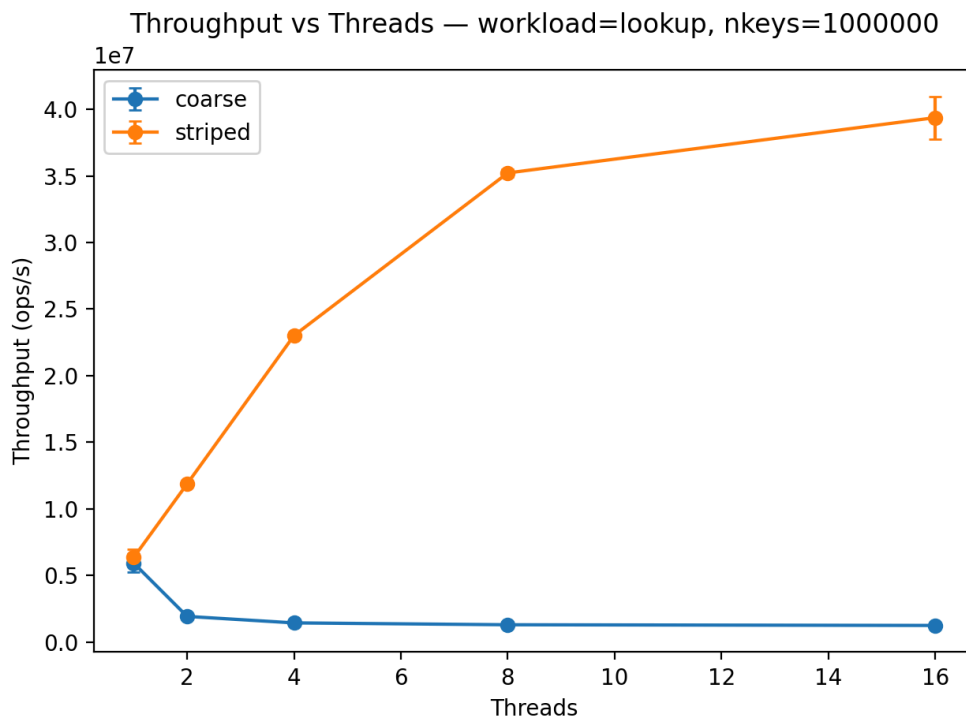
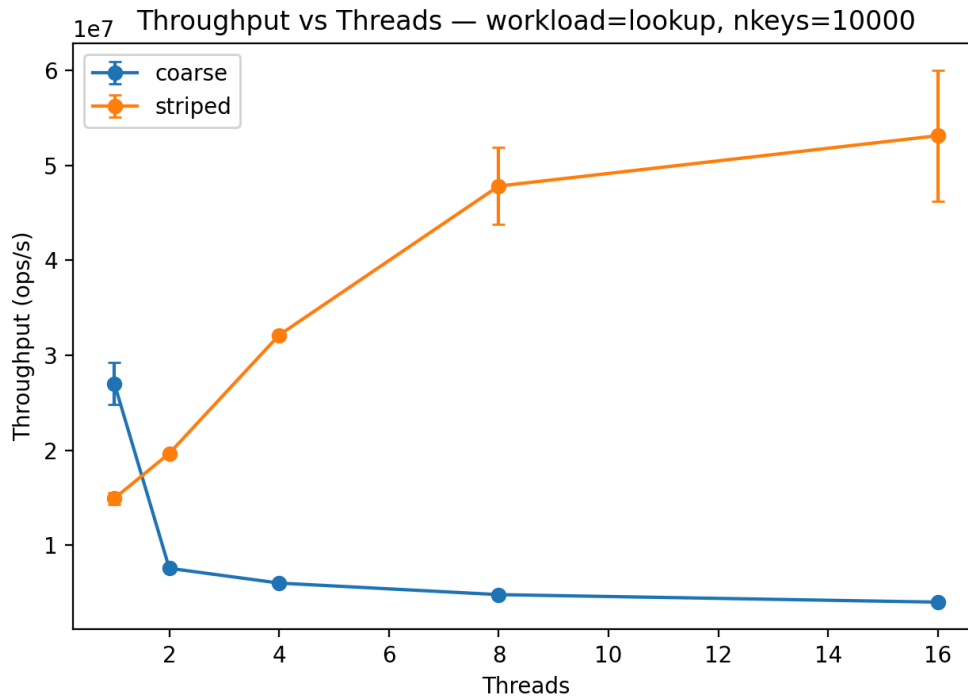
### Throughput Scaling: Lookup Workloads

Lookup-Only Throughput (10k Keys)

| Threads | Coarse (ops/s)      | Striped (ops/s)     |
|---------|---------------------|---------------------|
| 1       | $2.70e7 \pm 2.22e6$ | $1.49e7 \pm 6.33e5$ |
| 2       | $7.56e6 \pm 1.98e5$ | $1.97e7 \pm 1.10e5$ |
| 4       | $6.01e6 \pm 1.65e5$ | $3.21e7 \pm 1.04e5$ |
| 8       | $4.79e6 \pm 4.17e4$ | $4.78e7 \pm 4.07e6$ |
| 16      | $4.00e6 \pm 7.39e4$ | $5.31e7 \pm 6.91e6$ |

Lookup-Only Throughput (1M Keys)

| Threads | Coarse (ops/s)      | Striped (ops/s)     |
|---------|---------------------|---------------------|
| 1       | $5.93e6 \pm 7.09e5$ | $6.35e6 \pm 5.96e5$ |
| 2       | $1.93e6 \pm 3.57e4$ | $1.19e7 \pm 1.13e5$ |
| 4       | $1.44e6 \pm 3.54e4$ | $2.30e7 \pm 1.60e5$ |
| 8       | $1.30e6 \pm 1.18e4$ | $3.52e7 \pm 3.11e5$ |
| 16      | $1.25e6 \pm 1.29e4$ | $3.94e7 \pm 1.60e6$ |



The results show throughput as a function of thread count for lookup workloads at small and large table sizes. For both table sizes, the coarse grained implementation achieves its highest throughput with a single thread and shows decreasing performance as thread count increases. The striped locking implementation achieves higher throughput across all thread counts and improved scaling behavior. Throughput is lower for the larger table because more accesses

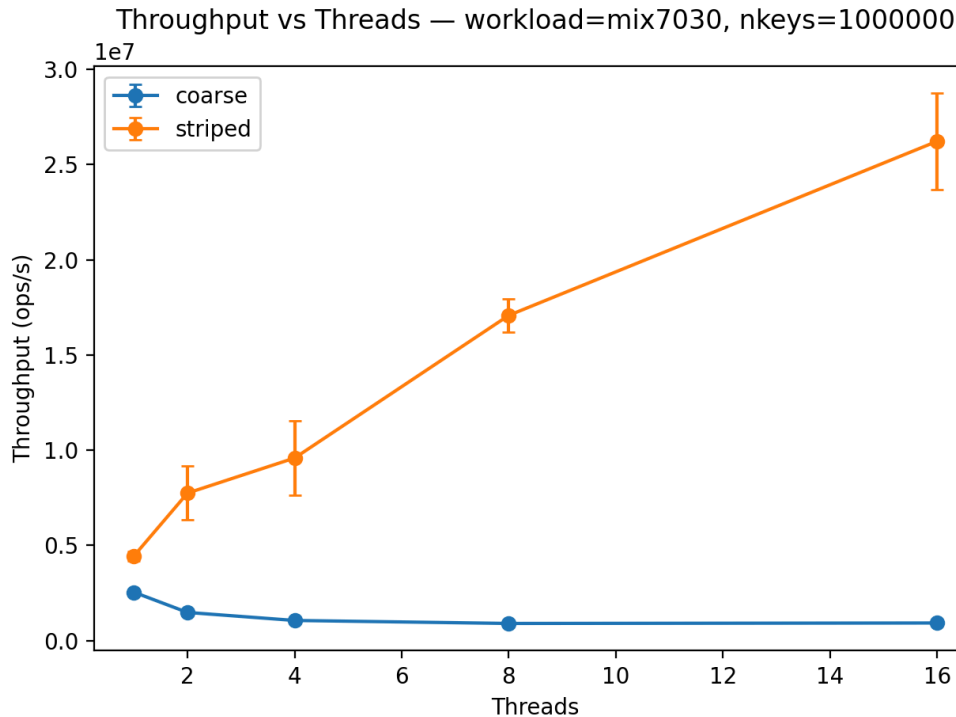
miss in cache and require main memory access which results in an increase in memory latency for both implementations.

The coarse grained implementation uses one global lock so all threads must wait for the same lock. As more threads are added, waiting time increases and performance drops. The striped implementation uses multiple locks which allows different buckets to be accessed at the same time. This reduces waiting and improves throughput. When the table is large more memory accesses are required which reduces performance for both implementations.

## Throughput Scaling: Mixed Workloads

Mixed Workload Throughput (70/30, 1M Keys)

| Threads | Coarse (ops/s)  | Striped (ops/s) |
|---------|-----------------|-----------------|
| 1       | 2.54e6 ± 4.05e4 | 4.45e6 ± 2.46e5 |
| 2       | 1.48e6 ± 2.74e4 | 7.75e6 ± 1.42e6 |
| 4       | 1.06e6 ± 3.98e4 | 9.59e6 ± 1.95e6 |
| 8       | 9.02e5 ± 3.69e4 | 1.71e7 ± 8.81e5 |
| 16      | 9.26e5 ± 2.16e4 | 2.62e7 ± 2.55e6 |



For the mixed workload, throughput decreases for both implementations as thread count increases. The coarse grained implementation degrades with additional threads. The striped implementation maintains higher throughput across all thread counts.

The mixed workload includes insert and erase operations, which require more time and cause additional memory writes. In the coarse grained design, these operations increase lock contention and force more serialization. Striped locking limits contention to smaller parts of the table which allows for more parallel work. However, performance still does not scale linearly because memory access and synchronization overhead increase at high thread counts.

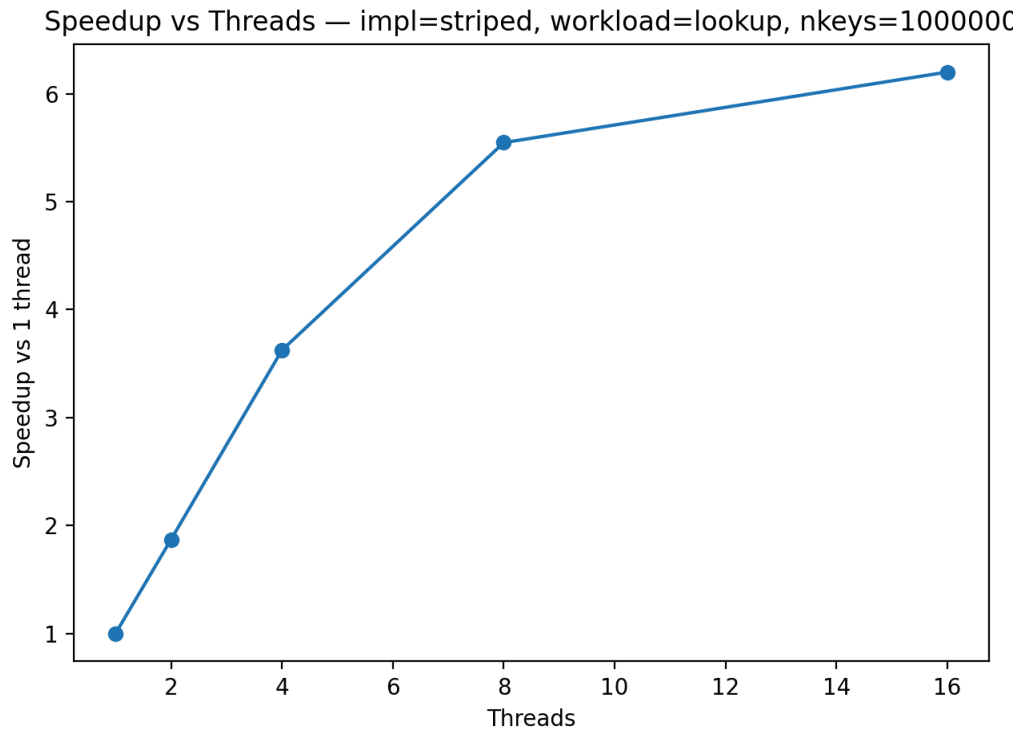
## Speedup Relative to Single Thread

Lookup Workload, 1M Keys Coarse Grained Locking

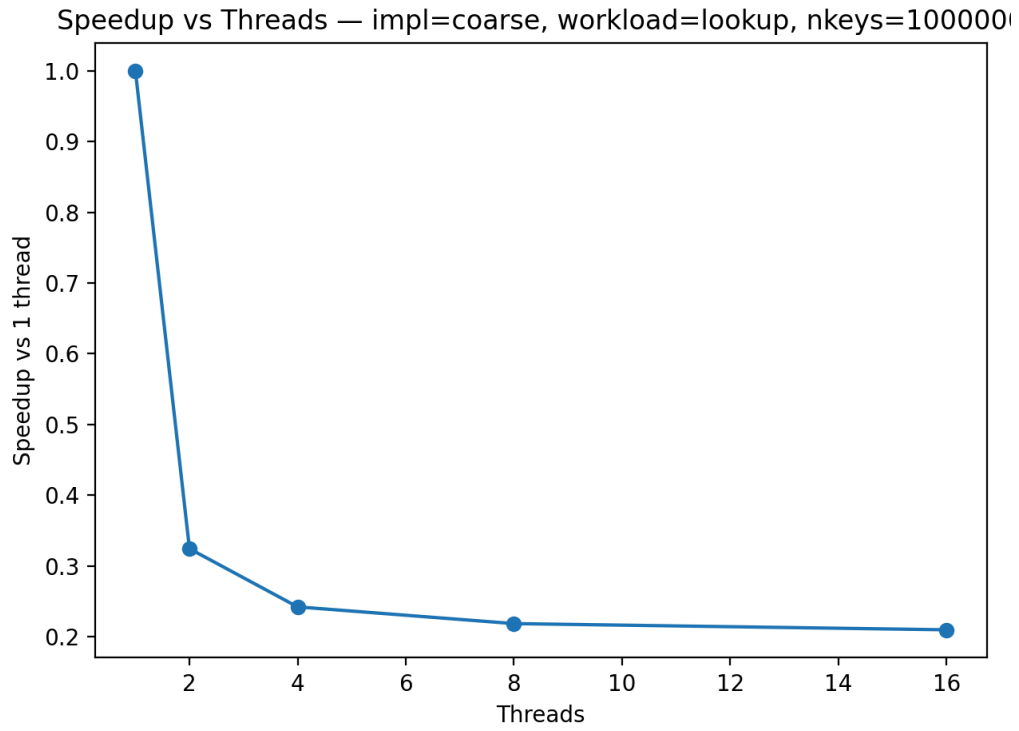
| Threads | Speedup |
|---------|---------|
| 1       | 1.00    |
| 2       | 0.32    |
| 4       | 0.24    |
| 8       | 0.22    |
| 16      | 0.21    |

## Lookup Workload 1M Keys Striped Locking

| Threads | Speedup |
|---------|---------|
| 1       | 1.00    |
| 2       | 1.87    |
| 4       | 3.62    |
| 8       | 5.55    |
| 16      | 6.20    |



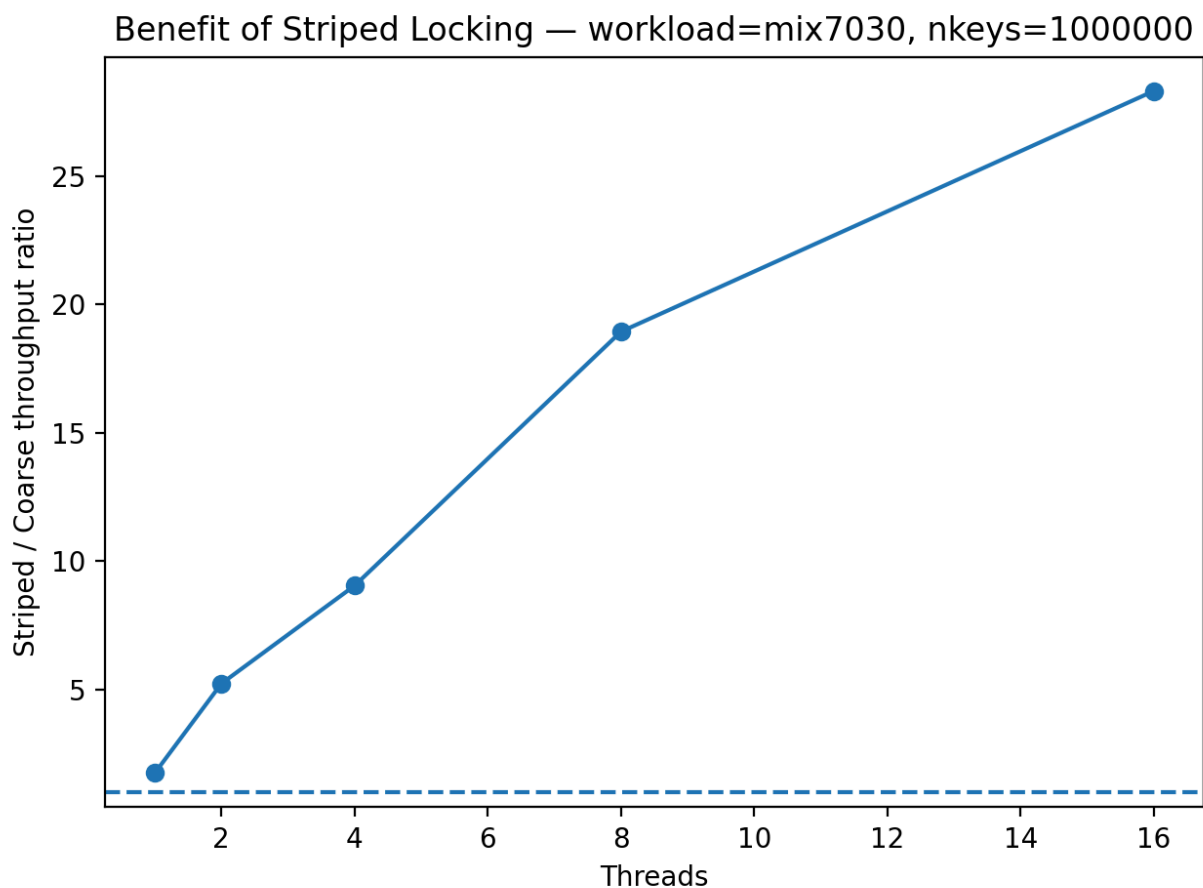




The coarse grained implementation shows speedup below one for all multi threaded cases which means performance gets worse when more threads are added. The striped implementation shows speedup as thread count increases with improvement slowing at higher thread counts.

In the coarse grained implementation, the global lock creates a large serial portion of execution. This prevents threads from running in parallel and causes performance to become worse. According to Amdahl's Law a large serial section limits scalability. Striped locking reduces the serial portion by spreading work across multiple locks which allows better parallel execution. Speedup is still limited by memory access and remaining synchronization costs.

## Benefit of Striped Locking



The throughput ratio between striped and coarse locking increases with thread count. At high thread counts, striped locking achieves higher throughput than coarse locking.

As more threads are added the cost of using a single global lock grows. Striped locking avoids this bottleneck by reducing contention and allowing concurrent access to different parts of the table. The increasing performance gap shows that finer grained synchronization is important for scalable performance in concurrent data structures.

## Conclusion

This project compared two synchronization strategies for a concurrent hash table being coarse grained locking and striped locking. The results show that coarse grained locking does not scale with increasing thread count due to high contention on a single global lock, whereas striped locking improves throughput by reducing contention and allowing parallel access to different parts of the table. Although striped locking does not achieve linear scaling because of memory and synchronization overhead it consistently outperforms the coarse grained approach across all workloads and table sizes. These results demonstrate that synchronization granularity plays an important role in the performance of concurrent data structures.

## References

- [1]OpenAI, "ChatGPT," chatgpt.com, Sep. 12, 2025. <https://chatgpt.com>
- [2]Grokikipedia, "Lock (computer science)," Grokikipedia, Jan. 21, 1970.  
[https://grokikipedia.com/page/Lock\\_\(computer\\_science\)](https://grokikipedia.com/page/Lock_(computer_science))
- [3]Wikipedia Contributors, "Concurrent hash table," Wikipedia, Apr. 08, 2025.
- [1]M. Michael, "High Performance Dynamic Lock-Free Hash Tables and List-Based Sets."  
Available: <https://docs.rs/crate/crossbeam/0.2.4/source/hash-and-skip.pdf>
- [1]"Summer 2007." [Online]. Available:  
<https://www3.cs.stonybrook.edu/~rezaul/Spring-2019/CSE613/reading/Amdahl-1967.pdf>